EPFL - Computer Science

CS-550 Formal Verification

Lecture 07 — November 11, 2026

Author: Jonathan Dutoit

Professor: Viktor Kuncak

Quantifier Elimination for Presburger Arithmetic

Formal Verification Methodology

Formal verification of program as a three-step approach:

1. Express properties in logic or code
   (assertions, preconditions, post-conditions, invariants, run-time error conditions)
2. Compile program meaning to **logical formulas**
   (verification-condition generator, symbolic execution)
3. Develop and use an automated theorem prover for generated conditions
   (SAT and SMT solving, resolution-based theorem proving, proof assistants)

Which logic to use?

Presburger arithmetic

Integer arithmetic with logical operations (and, or, not), quantifiers, only addition as arithmetic operations, and $<$ and $=$ as a relation.

- minimalistically one can define a variant of it over non-negative natural numbers as having $\wedge, \neg, \forall, +, =$ as the only symbols.

Definition. One-Point Rule: If $\bar{y}$ is a tuple of variables not containing x, then

$$\exists x.(x = t(\bar{y}) \wedge F(x, \bar{y})) \Leftrightarrow F(t(\bar{y}), \bar{y})$$

- *One point rule*: replaces LHS of equivalence by the RHS
- *Flattening*: used when t is complex, replaces RHS BY LHS.

By the same reasoning, we have **dual one-point rule for** $\forall$

$$\neg(\exists x.(x = t(\bar{y}) \wedge F(x, \bar{y})) \Leftrightarrow F(t(\bar{y}), \bar{y}))$$

which is equivalent to

$$\forall x.(x = t(\bar{y}) \rightarrow \neg F(x, \bar{y})) \Leftrightarrow \neg F(t(\bar{y}), \bar{y})$$

Hence we can deduce

$$\forall x.(x = t(\bar{y}) \rightarrow F(x, \bar{y})) \Leftrightarrow F(t(\bar{y}), \bar{y})$$

Quantifier Elimination (QE)

Given a formula $F(\bar{y})$ containing quantifiers, find a formula $G(\bar{y})$ **equivalent** to $F(\bar{y})$, that has **no quantifiers**, and has a **subset (or equal set) of free variables** of F.

- To check satisfiability of $H(\bar{y})$: eliminate the quantifiers from $\exists \bar{y}.H(\bar{y})$ and evaluate.
- Validity: eliminate quantifiers from $\forall \bar{y}.H(\bar{y})$ and evaluate.

Does Presburger Arithmetic admit QE?

Depends on the particular set of symbols! Quantifier elimination implies: if you can define some relationship between variables using an arbitrary, possibly quantified formula F, then you can also define same r using another quantifier-free formula G.

Example: $\exists y.x = y + y + y$ only have an equivalent formula without quantifiers if we have *divisibility.*

Presburger Arithmetic (PA)

$$\{+, -, =, <\} \cup \{K | K \in \mathbb{Z}\} \cup \{(K \cdot _) \mid K \in \mathbb{Z}\} \cup \{(K|_) \mid K \in \mathbb{Z}\}.$$

We also have, as usual: $\wedge, \vee, \neg, \rightarrow, \forall, \exists$.

- **Atomic formula**: a relation applied to argument.
  $t_1 = t_2$, $t_1 < t_2$, $K \mid t$
- **Literal**: Atomic formula or it's negation
  $\neg(x = y + 1)$

First, we consider the quantifier elimination problem of the form:

$$\exists y. \quad L_1 \wedge ... \wedge L_n$$

This will prove to be sufficient to eliminate all quantifiers!

The question becomes: can we eliminate $\exists$ from any **quantifier-free formula**?

$$\exists x.F(x,\bar{y})$$

where F is quantifier free, meaning that it only has $\wedge, \vee, \neg$ applied to atomic formulas. Convert F to **disjunctive normal form** where each C_i is a **conjunction of literals**.

$$\exists x.F \Leftrightarrow [\exists x. \bigvee_{i=1}^{m} C_i] \Leftrightarrow \bigvee_{i=1}^{m} (\exists x.C_i)$$

Nested Existential Quantifiers

Eliminate one quantifier at a time:

$$\begin{aligned} &\exists x_1.\exists x_2.\exists x_3.\, F_0(x_1,x_2,x_3,\bar{y}) \\ \rightsquigarrow\ &\exists x_1.\exists x_2.\, F_1(x_1,x_2,\bar{y}) \\ \rightsquigarrow\ &\exists x_1.\, F_2(x_1,\bar{y}) \\ \rightsquigarrow\ &F_3(\bar{y}) \end{aligned}$$

Each step eliminates $\exists$ from a conjunction of literals (after DNF).

Universal Quantifiers

Reduce $\forall$ to the already handled existential case:

$$\forall x.\, F_0(x,\bar{y}) \ \Leftrightarrow\ \neg[\exists x.\, \neg F_0(x,\bar{y})]$$

$\neg F_0$ is quantifier-free, so $\exists x.\, \neg F_0(x,\bar{y}) \rightsquigarrow F_1(\bar{y})$, and therefore

$$\forall x.\, F_0(x,\bar{y}) \ \rightsquigarrow\ \neg F_1(\bar{y})$$

Removing Any Alternation of Quantifiers

Push negations in so that only $\exists$ blocks remain, alternating with negations:

$$\begin{aligned} &\exists x_1.\, \forall x_2.\, \forall x_3.\, \exists x_4.\ F_0(x_1,x_2,x_3,x_4,\bar{y}) \\ \rightsquigarrow\ &\exists x_1.\, \neg\exists x_2.\, \exists x_3.\, \neg\exists x_4.\ F_0(x_1,x_2,x_3,x_4,\bar{y}) \\ \rightsquigarrow\ &\exists x_1.\, \neg\exists x_2.\, \exists x_3.\, \neg F_1(x_1,x_2,x_3,\bar{y}) \\ \rightsquigarrow\ &\exists x_1.\, \neg\exists x_2.\ F_2(x_1,x_2,\bar{y}) \\ \rightsquigarrow\ &\exists x_1.\, \neg F_3(x_1,\bar{y}) \\ \rightsquigarrow\ &F_4(\bar{y}) \end{aligned}$$

Each quantifier alternation involves one disjunctive normal form transformation. Note that for integers:

$$\neg(x<y) \Leftrightarrow y \le x \qquad x<y \Leftrightarrow x+1 \le y \qquad x \le y \Leftrightarrow x < y+1$$

Instead of $\le$ we chose to use $<$ only. We do not write $x > y$ but only $y < x$, since y is the variable being quantified.

Normal form of terms: All *terms* are build from $K, +, -, K\cdot_$, so using standard transformations they can be represented as: $K_0 + \sum_{i=1}^{n} K_i x_i$. We call such term a *linear term*.

Normal form for literals in PA:

$$\begin{aligned} &\neg(t_1 < t_2) \quad \text{becomes} \quad t_2 < t_1 + 1 \\ &\neg(t_1 = t_2) \quad \text{becomes} \quad t_1 < t_2 \vee t_2 < t_1 \\ &t_1 = t_2 \quad \text{becomes} \quad t_1 < t_2 + 1 \wedge t_2 < t_1 + 1 \quad (*) \\ &\neg(K \mid t) \quad \text{becomes} \quad \textstyle\bigvee_{i=1}^{K-1} K \mid t + i \\ &t_1 < t_2 \quad \text{becomes} \quad 0 < t_2 - t_1 \end{aligned}$$

To remove generated disjunctions, compute DNF again.
(*) Transformed equalities for simplicity. Usually, handle them directly.

Example: eliminate y from this formula

$$\exists y.\, 3y - 2w + 1 > -w \wedge 2y - 6 < z \wedge 4|5y + 1$$

Simplify/normalize using properties of integer operations:

$$\exists y. \quad 0 < -w + \mathbf{3y} + 1 \quad \wedge \quad 0 < \mathbf{\text{-}2y} + z + 6 \quad \wedge \quad 4 \,|\, \mathbf{5y} + 1$$

Least common multiple of coefficients $M = lcm(3,\ 2,\ 5) = 30$

$$\exists y. \quad 0 < -10w + \underline{\mathbf{30y}} + 10 \quad \wedge \quad 0 < \underline{\mathbf{\text{-}30y}} + 15z + 90 \quad \wedge \quad 24 \,|\, \underline{\mathbf{30y}} + 6$$

Let x denotes *30y*. It is **not an arbitrary x**. It is divisible by 30.

$$\exists x. \quad 0 < -10w + x + 10 \wedge 0 < -x + 15z + 90 \wedge 24 \,|\, x + 6 \wedge 30 \,|\, x$$

Lower and upper bounds

Consider the coefficient next to x in $0 < t$. If it is -1, move the term to left side. If it is 1, move the remaining terms to the left side. We obtain formula $F_1(x)$ of the form:

$$\bigwedge_{i=1}^{L} a_i < x \wedge \bigwedge_{j=1}^{U} x < b_j \wedge \bigwedge_{i=1}^{D} K_i \,|\, (x + t_i)$$

If there are no divisibility constraints ($D = 0$), the formula is equivalent to:

$$\max_i a_i + 1 \leq \max_j b_j - 1 \Leftrightarrow \bigwedge_{ij} a_i + 1 < b_j$$

Replacing variable by test terms

Common technique in quantifier elimination: replacing $F_1(x)$ with $\bigvee_k F_1(t_k)$, where t_k do not contain x. If $F_1(t_k)$ holds then certainly $\exists x.F_1(x)$. The example terms t_i when $D = 0$ and $L > 0$

$$\bigvee_{k=1}^{L} F_1(a_k + 1) \qquad \text{ensuring that at least one of them evalutes to } \max a_i + 1$$

and if $D > 0$, i.e., we have additional divisibility constraints:

$$\bigvee_{k=1}^{L} \bigvee_{i=1}^{N} F_1(a_k + i) \qquad \text{where N is the lcm of } K_1, ..., K_D$$

$$\exists x.F_1(x) \quad \equiv \quad \exists x. \, -10 + 10w < x \, \wedge \, x < 15z + 90 \, \wedge \, 24 \,|\, x + 6 \, \wedge \, 30 \,|\, x$$

Let's denote the lower bound $a_1 = -10w + 10$, $b_1 = 15z + 90$, $K_1 = 24, K_2 = 30$ hence lcm(K_1, K_2) $= 120$).

We replace with $a_1 + i < b_1$, meaning that x is replaced with $-10 + 10w + i$, which corresponds to:

$$\bigvee_{i=1}^{120} 10w + i < 100 + 15z \wedge 24 \,|\, 10w - 4 + i \wedge 30 \,|\, 10w - 10 + i$$

This is the solution. i is convenient written so that we avoid writing 120 disjuncts, but it's not a *variable*.

No lower bounds

Consider the case $L = 0$.

$$\bigwedge_{j=1}^{U} x < b_j \wedge \bigwedge_{i=1}^{D} K_i \,|\, (x + t_i)$$

We cannot try all lower bounds, as there are none. Thus, we can always find a x small enough to satisfy the upper bounds. Hence, we drop all constraints except divisibility, obtaining $F_2(x)$:

$$\bigwedge_{i=1}^{D} K_i \,|\, (x + t_i)$$

and then eliminate quantifier as

$$\bigvee_{i=1}^{N} F_2(i)$$

. **Example**

$$\exists x.\, x < 5 + 7z \wedge 2 \,|\, x \wedge 3 \,|\, x + 2$$

N = lcm(2, 3) = 6.

$$\bigvee_{i=1}^{6} (2 \,|\, i \wedge 3 \,|\, i + 2)$$

Summary

This complete the description of a quantifier elimination algorithm for Presburger Arithmetic (PA).

This algorithm and its correctness prove that:

- PA admits quantifier elimination
- Satisfiability, validity, entailment, equivalence of PA formulas is decidable.
  We can use the algorithm to prove verification conditions.
  Not the most efficient way, but gives insights on what can later be build to come up with better algorithms
- Quantified and quantifier-free formulas have the same expressive power.

Regardless of the procedure, cannot escape inherent computational complexity of deciding if quantified Presburger arithmetic formula is true, which has been characterized using alternating complexity classes by Bernman.

A set A is in *STA(s(n), t(n), a(n))* iff there exists a single-tape *alternating* Turing machine M_i (making $\wedge$ as well as $\vee$ choices) that accepts A and runs withing space *s(n)*, time *t(n)* and performs at most *a(n)* alternations between $\wedge$ and $\vee$ choices.

Deciding of a PA formula of size n with m quantifiers is in

$$STA(*, 2^{n^{O(m)}}, m)$$

Complexity is particularly sensitive to m.